

BLE communication integration for automotive systems using esp32.

PC Server – ESP32 Client :

1. Introduction

Bluetooth Low Energy (BLE) is a wireless communication technology designed for low-power communication between electronic devices. It is widely used in embedded and automotive applications where devices need to exchange small amounts of data efficiently.

The objective is to establish BLE communication between a PC Server and an ESP32 Client and to demonstrate Read, Write, Notify, and basic BLE security functions.

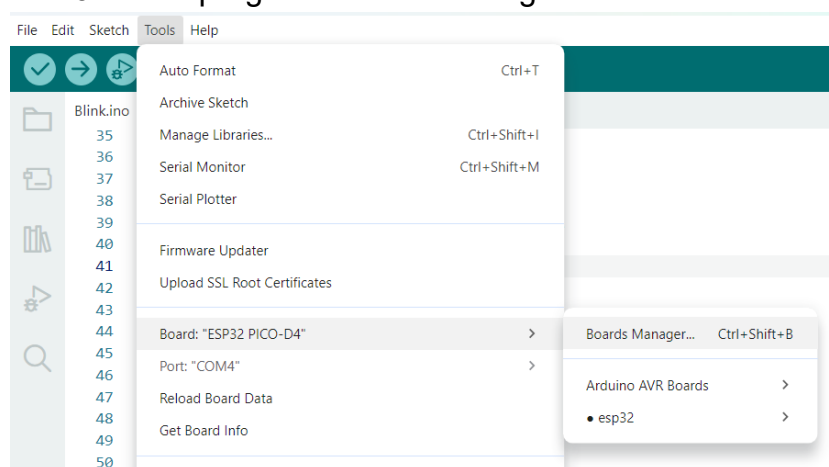
2. BLE Environment Setup

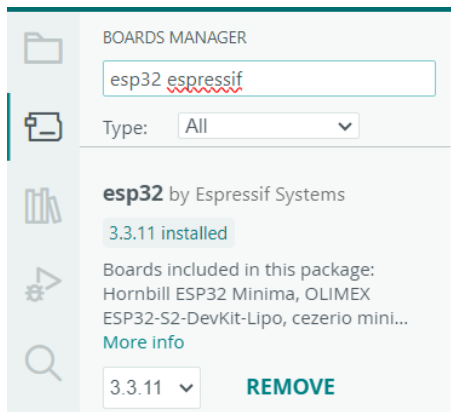
2.1 Required Hardware

- ESP32
- USB cable
- LED
- Resistor
- Push button
- PC

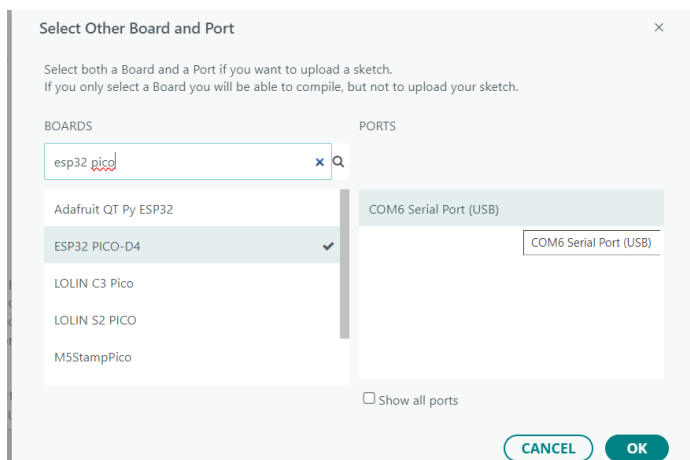
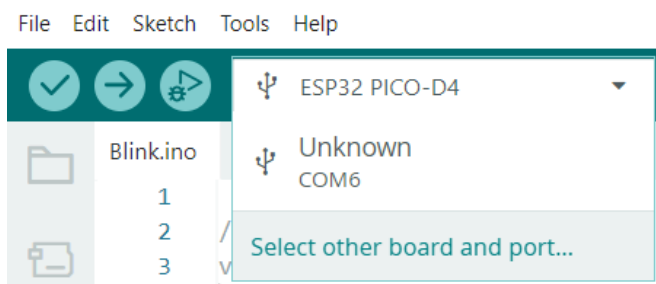
2.2 ESP32 Software Setup

The ESP32 board support package was installed in the Arduino IDE to allow the ESP32 to be programmed and configured.





The appropriate ESP32 board was selected in the Arduino IDE, together with the corresponding serial communication port.



2.3 PC Software Setup

The PC Server is implemented in Python using the Bless (Bluetooth Low Energy Server Supplement) BLE library.

```
import asyncio
import time
from datetime import datetime
from bless import (BlessServer, BlessGATTCharacteristic, GATTCharacteristicProperties, GATTAttributePermissions) # to install: pip install bless

SERVER_NAME = "PC"
SERVICE_UUID = ("b4250400-3446-4654-b59a-1c888d447433")
CHARACTERISTIC_UUID = ("b4250401-3446-4654-b59a-1c888d447433")
```

PC BLE SERVER

```
-----  
Local Date & Time : 2026-08-31 11:28:14  
Device Name       : PC  
Device ID        : 01  
Role             : SERVER  
Service UUID     : b4250400-3446-4654-b59a-1c888d447433  
Characteristic   : b4250401-3446-4654-b59a-1c888d447433  
Diagnostic Handle : 0x002A
```

3. Configuring the PC as a BLE Server

The PC uses BlessServer to create the BLE Server.

```
my_server = BlessServer(name=SERVER_NAME)
```

The server is started with `my_server.start()`.

```
await my_server.start()
```

3.1 Connection Handling

When the server receives its first relevant request, `connection_established()` verifies whether a connection has already been established.

```
def connection_established():  
  
    global connection_message_shown  
    global security_message_shown  
  
    if connection_message_shown:  
        return  
  
    connection_message_shown = True  
  
    print()  
    print("[BLE] CONNECTION ESTABLISHED")  
    print("[BLE] ESP connected to PC.")  
    print()
```

```
[BLE] Server started successfully.  
[BLE] Waiting for Client...
```

```
11:28:14.677 PC TX GAP ADV_IND - PC_server advertising B4 25 04 00 .%.  
11:28:15.684 PC TX GAP ADV_IND - PC_server advertising B4 25 04 00 .%.  
[BLE] CONNECTION ESTABLISHED  
[BLE] ESP connected to PC.
```

3.2 Disconnection Handling

The ESP32 sends 0x00 every 2 seconds while connected. If no GATT activity is received by the PC for more than 5 seconds, the watchdog reports a connection loss and the PC restarts advertising after disconnection.

```

async def connection_watchdog():

    global connection_active
    global security_message_shown

    while True:

        await asyncio.sleep(1)

        if not connection_active:
            continue

        last = (time.time()- last_packet_time) # time since last packet received
        if last > 5.0:
            print()
            print("[BLE] CONNECTION LOST")
            print("[BLE] No activity received for " f"{last:.1f} seconds.")

            connection_active = False
            security_message_shown = False
            print("[BLE] Waiting for Connection...")

```

```

[BLE] CONNECTION LOST
[BLE] No GATT activity received for 5.8 seconds.
[BLE] Waiting for Connection...

```

4. BLE Advertising

The **advertising()** callback function represents the advertising phase of the PC Server. While no connection has been established, the application periodically generates an **ADV_IND** log entry identifying the PC as a server.

```

connection_active = False
Windsurf: Refactor | Explain | Generate Docstring
async def advertising():

    global connection_active

    advertising_payload = bytes.fromhex("B4 25 04 00" )
    while True:
        if not connection_active:
            log_event("TX", "GAP", "ADV_IND", advertising_payload, advertising_payload, None, "PC_server advertising",)
            await asyncio.sleep(1)

```

```

11:28:14.677 PC TX GAP ADV_IND - PC_server advertising B4 25 04 00 .%.
11:28:15.684 PC TX GAP ADV_IND - PC_server advertising B4 25 04 00 .%.

```

5. BLE Services and Characteristics

The PC uses Bless to create custom services, and custom characteristics.

5.1 Custom Service/ Characteristic UUIDs

The UUIDs identify the custom service and characteristic used by the ESP32 during GATT discovery.

Create custom service UUID + characteristic UUID using UUID generator
(<https://www.uuidgenerator.net/>)

```
SERVICE_UUID = "b4250400-3446-4654-b59a-1c888d447433"  
CHARACTERISTIC_UUID = "b4250401-3446-4654-b59a-1c888d447433"
```

5.2 BLE Service

```
await my_server.add_new_service(SERVICE_UUID)
```

5.3 BLE Characteristic

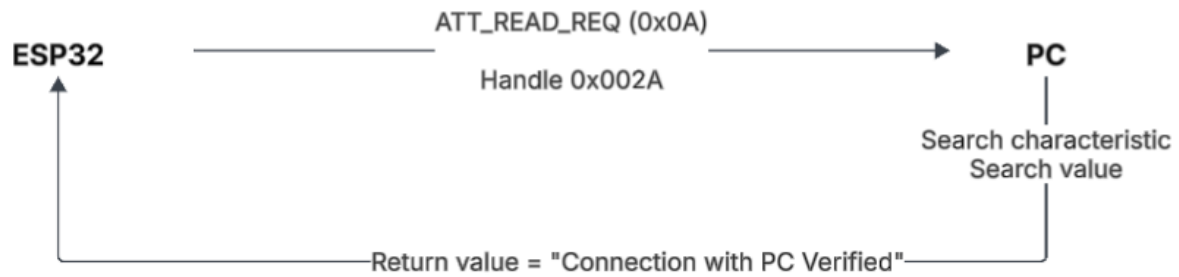
The characteristic supports three operations: Read, Write, and Notify.

```
properties = (GATTCharacteristicProperties.write| GATTCharacteristicProperties.read| GATTCharacteristicProperties.notify)  
permissions = (GATTAttributePermissions.writeable| GATTAttributePermissions.readable)  
await my_server.add_new_characteristic(SERVICE_UUID,CHARACTERISTIC_UUID,properties,None,permissions,)
```

6. Characteristic Read Operation

The ESP32 uses the BOOT button to trigger a Read operation. The PC returns the value "Connection with PC Verified".

```
def read_request(characteristic: BlessGATTCharacteristic, **kwargs):  
  
    global connection_active  
    global last_packet_time  
  
    connection_active = True  
    last_packet_time = time.time()  
  
    connection_established()  
  
    # Read request  
    pdu = build_read_request_pdu(DIAGNOSTIC_HANDLE)  
    l2cap_pdu = build_l2cap_pdu(pdu)  
    log_event("RX","L2CAP","DATA",l2cap_pdu,b"",None,"Length=%d, CID=0x0004" % len(pdu),)  
    log_event("RX","ATT","ATT_READ_REQ",pdu,b"",DIAGNOSTIC_HANDLE,"Read request",)  
    value = b"Connection with PC Verified"  
    characteristic.value = value  
  
    # READ RESPONSE  
    response_pdu = build_read_response_pdu(value)  
    l2cap_pdu = build_l2cap_pdu(response_pdu)  
    log_event("TX","L2CAP","DATA",l2cap_pdu, value,None,"Length=%d, CID=0x0004" % len(response_pdu),True,)  
    log_event("TX","ATT","ATT_READ_RSP",response_pdu,value,DIAGNOSTIC_HANDLE,"Read response",True,)  
    return value
```



11:28:18.634	PC	RX	L2CAP	DATA - Length=3, CID=0x0004	03 00 04 00 0A 2A 00	-
11:28:18.634	PC	RX	ATT	ATT_READ_REQ - Read request	0A 2A 00	-
11:28:18.634	PC	TX	L2CAP	DATA - Length=28, CID=0x0004	1C 00 04 00 0B 43 6F 6E 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64	Connection with PC Verified
11:28:18.634	PC	TX	ATT	ATT_READ_RSP - Read response	0B 43 6F 6E 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64	Connection with PC Verified

7. Characteristic Write Operation

A TOGGLE_LOCK write changes the simulated vehicle state. The new state is then sent to the ESP32 by notification.

```

def write_request(characteristic: BlessGATTCharacteristic,value: bytes,**kwargs):

    global connection_active
    global last_packet_time
    global car_is_locked

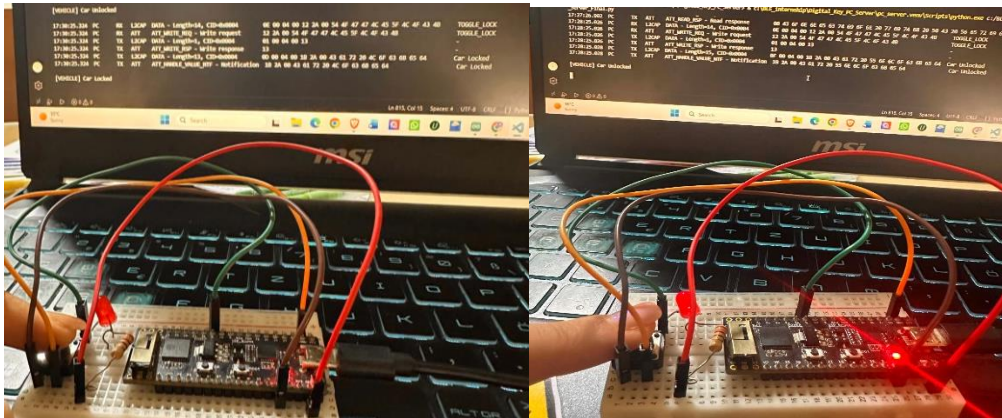
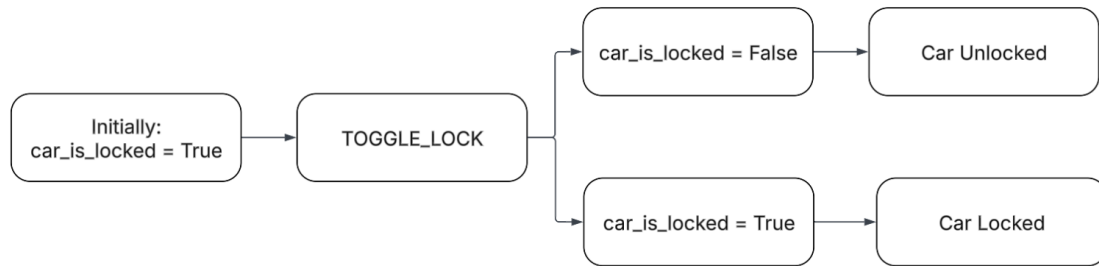
    connection_active = True
    last_packet_time = time.time()

    connection_established()

    if value == b"\x00":
        return
    command = value.decode("utf-8",errors="replace") # decode the received bytes to a string
    # WRITE REQUEST
    request_pdu = build_write_request_pdu(DIAGNOSTIC_HANDLE,value)
    l2cap_pdu = build_l2cap_pdu(request_pdu)
    log_event("RX","L2CAP","DATA",l2cap_pdu,value,None,"Length=%d, CID=0x0004" % len(request_pdu),)
    log_event("RX","ATT","ATT_WRITE_REQ",request_pdu,value,DIAGNOSTIC_HANDLE,"Write request",)
    # WRITE RESPONSE
    response_pdu = build_write_response_pdu()
    l2cap_pdu = build_l2cap_pdu(response_pdu)
    log_event("TX","L2CAP","DATA",l2cap_pdu,b"",None, "Length=%d, CID=0x0004" % len(response_pdu),)
    log_event("TX","ATT","ATT_WRITE_RSP",response_pdu,b"",DIAGNOSTIC_HANDLE,"Write response",)
    # APPLICATION LOGIC
    if command == COMMAND_TOGGLE_LOCK:
        car_is_locked = not car_is_locked
        if car_is_locked: status = CAR_LOCKED
        else: status = CAR_UNLOCKED

        if main_loop is not None:
            asyncio.run_coroutine_threadsafe(send_notification(status, 1.5),main_loop,)
    else:
        print(f"[APP] Unknown command: {command}")
  
```

11:28:25.760	PC	RX	L2CAP	DATA - Length=14, CID=0x0004	0E 00 04 00 12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B	TOGGLE_LOCK
11:28:25.760	PC	RX	ATT	ATT_WRITE_REQ - Write request	12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B	TOGGLE_LOCK
11:28:25.760	PC	TX	L2CAP	DATA - Length=1, CID=0x0004	01 00 04 00 13	-
11:28:25.760	PC	TX	ATT	ATT_WRITE_RSP - Write response	13	-



8. Characteristic Notify Operation

After TOGGLE_LOCK, the PC updates the characteristic value and sends the new vehicle state. The ESP32 receives the notification and updates the LED.

TOGGLE_LOCK -> Change car_is_locked ("Car Locked" / "Car Unlocked") ->
Update characteristic value -> Notification -> ESP32 Client.

```

async def send_notification(message: str):

    if my_server is None:
        return

    characteristic = (my_server.get_characteristic(Characteristic_UUID))

    if characteristic is None:
        print("[ERROR] Characteristic not found.")
        return

    value = message.encode("utf-8")

    characteristic.value = value

    my_server.update_value( SERVICE_UUID, Characteristic_UUID,)

    # NOTIFICATION PDU
    notification_pdu = build_notification_pdu(DIAGNOSTIC_HANDLE,value)

    l2cap_pdu = build_l2cap_pdu(notification_pdu)
    log_event("TX", "L2CAP", "DATA", l2cap_pdu,value, None, "Length=%d, CID=0x0004" % len(notification_pdu),)
    log_event("TX", "ATT", "ATT_HANDLE_VALUE_NTF", notification_pdu,value,DIAGNOSTIC_HANDLE,"Notification",)

    print()
    print(f"[VEHICLE] {message}")
    print()
  
```

```

11:28:25.761 PC TX L2CAP DATA - Length=15, CID=0x0004 0F 00 04 00 1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked
11:28:25.761 PC TX ATT ATT_HANDLE_VALUE_NTF - Notification 1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked
[VEHICLE] Car Unlocked

11:28:28.205 PC TX L2CAP DATA - Length=13, CID=0x0004 0D 00 04 00 1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked
11:28:28.206 PC TX ATT ATT_HANDLE_VALUE_NTF - Notification 1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked
[VEHICLE] Car Locked

```

9. Security (Passkey Entry)

The PC uses the Windows Bluetooth stack through Bless. Because the application does not expose the raw SMP authentication-complete event, the authentication-success message is displayed at the point where the link becomes operational.

The ESP32 is configured for passkey pairing, MITM protection, and secure connections. BLE security procedures are handled by the Bluetooth stacks.

```

def on_subscribe(characteristic: BlessGATTCharacteristic, session):

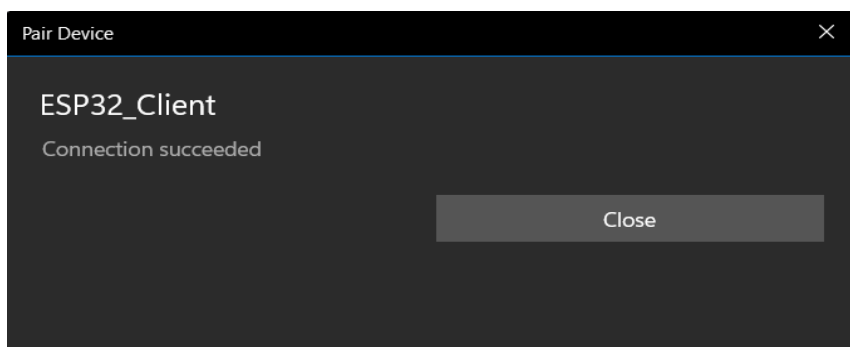
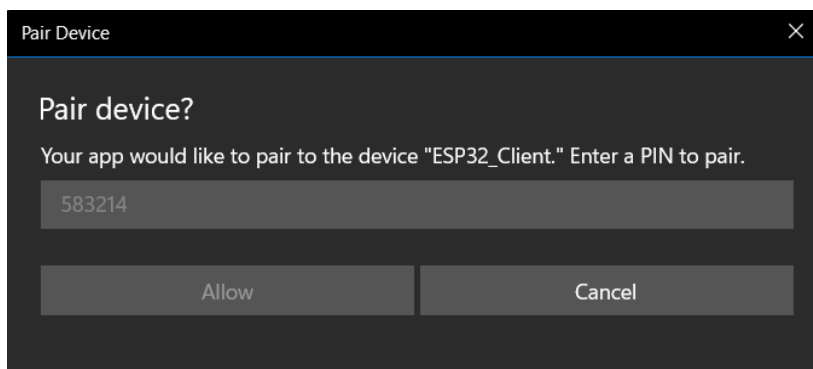
    global connection_active
    global last_packet_time
    global security_message_shown

    connection_active = True
    last_packet_time = time.time()

    if not security_message_shown:
        security_message_shown = True

    print()
    print("[SMP] AUTHENTICATION SUCCESS")
    print("[SMP] BLE security established.")
    print()

```



```
[BLE] CONNECTION ESTABLISHED
[BLE] ESP connected to PC.

[SMP] AUTHENTICATION STARTED
[SMP] Passkey: 583214
[SMP] Waiting for passkey confirmation...

[SMP] AUTHENTICATION SUCCESS
[SMP] BLE security established.
```

10. BLE log

PC Server:

```
PC BLE SERVER
-----
Local Date & Time : 2026-08-31 11:28:14
Device Name      : PC
Device ID       : 01
Role            : SERVER
Service UUID    : b4250400-3446-4654-b59a-1c888d447433
Characteristic : b4250401-3446-4654-b59a-1c888d447433
Diagnostic Handle : 0x002A

SECURITY
-----
BLE security      : Windows BLE stack
Pairing          : Windows BLE pairing
Authentication    : BLE SMP
MITM protection  : Enabled
-----

TIME            DEVICE DIR  LAYER EVENT                                PDU HEX                                PAYLOAD ASCII
-----
[BLE] Server started successfully.
[BLE] Waiting for Client...

11:28:14.677 PC    TX   GAP   ADV_IND - PC_server advertising         B4 25 04 00                            .%.
11:28:15.684 PC    TX   GAP   ADV_IND - PC_server advertising         B4 25 04 00                            .%.

[BLE] CONNECTION ESTABLISHED
[BLE] ESP connected to PC.

[SMP] AUTHENTICATION STARTED
[SMP] Passkey: 583214
[SMP] Waiting for passkey confirmation...

[SMP] AUTHENTICATION SUCCESS
[SMP] BLE security established.
```

```
[BLE] Server started successfully.
[BLE] Waiting for Client...

11:28:14.677 PC    TX   GAP   ADV_IND - PC_server advertising         B4 25 04 00                            .%.
11:28:15.684 PC    TX   GAP   ADV_IND - PC_server advertising         B4 25 04 00                            .%.

[BLE] CONNECTION ESTABLISHED
[BLE] ESP connected to PC.

[SMP] AUTHENTICATION STARTED
[SMP] Passkey: 583214
[SMP] Waiting for passkey confirmation...

[SMP] AUTHENTICATION SUCCESS
[SMP] BLE security established.

11:28:18.634 PC    RX   L2CAP DATA - Length=3, CID=0x0004      03 00 04 00 0A 2A 00                    -
11:28:18.634 PC    RX   ATT   ATT_READ_REQ - Read request         0A 2A 00                                -
11:28:18.634 PC    TX   L2CAP DATA - Length=28, CID=0x0004      1C 00 04 00 0B 43 6F 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64 Connection with PC Verified
11:28:18.634 PC    TX   ATT   ATT_READ_RSP - Read response        0B 43 6F 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64 Connection with PC Verified
11:28:25.760 PC    RX   L2CAP DATA - Length=14, CID=0x0004      0E 00 04 00 12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
11:28:25.760 PC    RX   ATT   ATT_WRITE_REQ - Write request         12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
11:28:25.760 PC    TX   L2CAP DATA - Length=1, CID=0x0004      01 00 04 00 13                            -
11:28:25.760 PC    TX   ATT   ATT_WRITE_RSP - Write response        13                            -
11:28:25.761 PC    TX   L2CAP DATA - Length=15, CID=0x0004      0F 00 04 00 1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked
11:28:25.761 PC    TX   ATT   ATT_HANDLE_VALUE_NTF - Notification    1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked

[VEHICLE] Car Unlocked

11:28:28.205 PC    RX   L2CAP DATA - Length=14, CID=0x0004      0E 00 04 00 12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
11:28:28.205 PC    RX   ATT   ATT_WRITE_REQ - Write request         12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
11:28:28.205 PC    TX   L2CAP DATA - Length=1, CID=0x0004      01 00 04 00 13                            -
11:28:28.205 PC    TX   ATT   ATT_WRITE_RSP - Write response        13                            -
11:28:28.205 PC    TX   L2CAP DATA - Length=13, CID=0x0004      0D 00 04 00 1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked
11:28:28.206 PC    TX   ATT   ATT_HANDLE_VALUE_NTF - Notification    1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked

[VEHICLE] Car Locked
```

ESP Client:

```
ESP32 BLE CLIENT
-----
Device Name      : ESP32_Client
Device ID       : 10
Role            : BLE_CLIENT
Service UUID    : b4250400-3446-4654-b59a-1c888d447433
Characteristic  : b4250401-3446-4654-b59a-1c888d447433
Diagnostic Handle: 0x002A

SECURITY
-----
BLE security     : ESP32 BLE stack
Pairing         : Passkey pairing
Authentication   : BLE SMP
MITM protection : Enabled
-----

TIME            DEVICE DIR  LAYER  EVENT                                PDU HEX                                PAYLOAD ASCII
-----
[Gap] Scanning for PC_server...

[Gap] PC_server discovered.
+0009.424      ESP32  RX   GAP    ADV_IND - PC_server advertising detected -
-

[BLE] Connecting to PC_server...

[BLE] CONNECTION ESTABLISHED
[BLE] Connected to PC_server.

[BLE] Connected to PC_server.

+0009.775      ESP32  INT  SMP    AUTHENTICATION - CID 0x0006 SUCCESS -
-

[SMP] AUTHENTICATION SUCCESS
[SMP] BLE security established.

+0011.123      ESP32  RX   GATT    SERVICE_FOUND - b4250400-3446-4654-b59a-1c888d447433 -
+0011.138      ESP32  RX   GATT    CHARACTERISTIC_FOUND - b4250401-3446-4654-b59a-1c888d447433 -
-

[SMP] Waiting for BLE authentication...
+0011.151      ESP32  TX   GATT    NOTIFY_ENABLE - Notification subscription -
-
+0013.351      ESP32  TX   L2CAP   DATA - CID 0x0004                                03 00 04 00 0A 2A 00                                -
+0013.362      ESP32  TX   ATT     ATT_READ_REQ - Read request                                0A 2A 00                                -
+0013.397      ESP32  RX   L2CAP   DATA - Length=26, CID=0x0004                                1C 00 04 00 08 43 6F 6E 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64 Connection with F
+0013.414      ESP32  RX   ATT     ATT_READ_RSP - Read response                                08 43 6F 6E 6E 65 63 74 69 6F 6E 20 77 69 74 68 20 50 43 20 56 65 72 69 66 69 65 64 Connection with PC Verified
+0020.466      ESP32  TX   L2CAP   DATA - Length=14, CID=0x0004                                0E 00 04 00 12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
+0020.478      ESP32  TX   ATT     ATT_WRITE_REQ - Write request                                12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
+0020.523      ESP32  RX   L2CAP   DATA - Length=15, CID=0x0004                                0F 00 04 00 1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked
+0020.538      ESP32  RX   ATT     ATT_HANDLE_VALUE_NTF - Notification                                1B 2A 00 43 61 72 20 55 6E 6C 6F 63 6B 65 64 Car Unlocked
+0020.551      ESP32  RX   L2CAP   DATA - Length=1, CID=0x0004                                01 00 04 00 13 -
+0020.562      ESP32  RX   ATT     ATT_WRITE_RSP - Write response                                13 -
+0022.911      ESP32  TX   L2CAP   DATA - Length=14, CID=0x0004                                0E 00 04 00 12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
+0022.924      ESP32  TX   ATT     ATT_WRITE_REQ - Write request                                12 2A 00 54 4F 47 47 4C 45 5F 4C 4F 43 4B TOGGLE_LOCK
+0022.968      ESP32  RX   L2CAP   DATA - Length=13, CID=0x0004                                0D 00 04 00 1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked
+0022.982      ESP32  RX   ATT     ATT_HANDLE_VALUE_NTF - Notification                                1B 2A 00 43 61 72 20 4C 6F 63 6B 65 64 Car Locked
+0022.995      ESP32  RX   L2CAP   DATA - Length=1, CID=0x0004                                01 00 04 00 13 -
+0023.007      ESP32  RX   ATT     ATT_WRITE_RSP - Write response                                13 -
```